

ADR-0003 — Server Topology: Modular Monolith vs Microservices

Field	Value
ADR	0003
Title	Server topology: modular monolith vs microservices
Revision	2
Created	2026-06-07
Last modified	2026-06-07
Status	Proposed
Status summary	Decides the MVP deployment topology for the Helix OTA Go control plane and the concrete trigger to split into independent services. Resolves contradiction C2 from ../additions_synthesis.md §5.
Decision	Modular monolith for 1.0.0-MVP — one deployable Go binary with enforced internal package boundaries that mirror future service seams; rollout-engine and OS-adapter designed as extractable modules. Split to services only when a defined scale trigger fires.
Deciders	Lead architect (synthesis); operator review gate pending.
Scope	The Helix-authored Go control plane only. Wrapped third-party back ends (e.g. hawkBit, pending ADR-0001) are separate deployables regardless of this decision.
Locked decisions honored	Native Android A/B (AOSP <code>update_engine</code>) + custom Go control plane; research decides the wrap target (ADR-0001); mandated stack (Go + Gin + Brotli + HTTP/3→HTTP/2, REST-primary; PostgreSQL + MinIO/S3).
Anti-bluff rule	Every claim traces to the landscape report, a stack note, or the additions synthesis (cited by slug/section). Items

Field	Value
	the sources flagged as unverified are carried forward as UNVERIFIED . This ADR introduces no facts not present in the cited sources.
Supersedes / Superseded by	—
Fixed (rev 2)	Anti-bluff pass: softened the telemetry-vs-rollout volume asymmetry to UNVERIFIED and dropped its source attribution; corrected the DMF/RabbitMQ-deferral citations from eclipse-hawkbite §4 to eclipse-hawkbite §7 + ota_landscape_report §4; flagged the broken OS-adapter destination (00-master/architecture.md) inherited from additions_synthesis §4 — the master design file is 2026-06-07-helix-ota-design.md (architecture is its §4), pending upstream correction.
Related	ADR-0001 (wrapped engine), ADR-0002 (supply-chain trust), ADR-0004 (transport), ADR-0005 (delta updates).

Table of contents

- [1. Context](#)
- [2. Options considered](#)
 - [2.1 Option A — Microservices from MVP](#)
 - [2.2 Option B — Modular monolith \(extractable seams\)](#)
 - [2.3 Option C — Unstructured single-binary monolith](#)
- [3. Decision](#)
 - [3.1 Module boundaries \(the seams\)](#)
 - [3.2 Scale trigger to split into services](#)
- [4. Consequences](#)
 - [4.1 Positive](#)
 - [4.2 Negative](#)
- [5. Status](#)
- [6. Compliance notes \(HelixConstitution\)](#)
- [7. Open / UNVERIFIED items](#)
- [8. Sources](#)

1. Context

The two operator-supplied input drafts disagree on server topology. `initial_research_02.md` proposes a **microservices** topology with a pluggable OS-adapter architecture; `initial_research.md` proposes a leaner Go server that orchestrates the wrapped engine. The synthesis recorded this as contradiction **C2** and routed the final verdict to this ADR. [additions_synthesis §2, §5 C2, §7]

The governing constraints are fixed before this decision:

- **Locked strategy:** native Android A/B (AOSP `update_engine`) + a **custom Go control plane**, wrapping OSS only where it adds value. [ota_landscape_report §metadata; additions_synthesis §1]
- **Mandated stack:** Go + Gin + Brotli + HTTP/3→HTTP/2 (REST-primary); PostgreSQL + MinIO/S3. [ota_landscape_report §metadata; additions_synthesis §1, §5 C4]
- **Phase-1 target:** Android 15 first (Orange Pi 5 Max / RK3588 class); Linux/universal is a later phase. [ota_landscape_report §metadata]

Two architectural facts from the foundation corpus pre-shape the topology:

1. The wrapped-engine choice (ADR-0001) may introduce **hawkBit as a separate JVM/Spring-Boot deployable** sitting *behind* the Go control plane (never public). If adopted, hawkBit already constitutes a second process, so the JVM runtime is the major operational cost called out — independent of whether the Helix-authored code is mono or micro. [ota_landscape_report §3.2; eclipse-hawkbit]
2. The landscape report explicitly recommends the **leanest MVP topology** for the wrapped back end: hawkbit-monolith + PostgreSQL + DDI-only (no RabbitMQ unless DMF push is needed) + MinIO/S3. The report’s own bias is toward *fewer moving parts* at MVP. [ota_landscape_report §3.2, §4 (DMF/RabbitMQ deferral); eclipse-hawkbit §7]

The decoupling requirement is also fixed by the master design: artifact-validator, rollout-engine, ota-protocol types, the `update_engine` bridge, and the OS-adapter are to be **separate, independently testable modules** so future OSes and future projects can reuse them. This requires *internal modularity* but does not, by itself, mandate *separate deployables*. [master-design §4 decoupling principle (§11.4.28)]

The synthesis pre-resolved C2 toward a modular monolith (“one deployable, internal package boundaries mirroring the future services ... the OS-adapter + rollout-engine seams stay extractable; microservices revisited at scale”). This ADR ratifies that direction with evidence and pins the concrete scale trigger. [additions_synthesis §5 C2]

2. Options considered

2.1 Option A — Microservices from MVP

Decompose the Helix-authored control plane into independently deployed services (e.g. release/artifact service, rollout/campaign service, telemetry service, device-API gateway) from day one, as drafted in `initial_research_02.md`. [additions_synthesis §2, §5 C2]

Evidence for. - The staged-rollout engine is inherently OS-agnostic and is already specified as an extractable submodule, so a service boundary there is natural. [additions_synthesis §4 (rollout engine → 1.0.1-staged-rollout/rollout_engine.md, “engine is OS-agnostic submodule”)] - Mender — the only studied stack that ships a comparable open back end — is itself a Go **microservices** server (Mongo/NATS/Traefik), demonstrating the pattern is viable for OTA at scale. [mender; ota_landscape_report §2.3 mender Mechanism]

Evidence against. - Mender's microservices stack (Mongo/NATS/Traefik) is cited as a *conflict* with the mandated Gin + PostgreSQL + MinIO + REST stack — its topology is a cost, not a model to copy wholesale. [ota_landscape_report §2.3 mender Go-fit (2), Wrap-ability (1); §4 Mender row] - The report's leanest-MVP recommendation deliberately *minimizes* runtimes and brokers (DDI-only, no RabbitMQ). Adding N Helix services on top of a possible JVM hawkBit multiplies operational surface against that grain. [ota_landscape_report §3.2, §4 (DMF/RabbitMQ deferral); eclipse-hawkbit §7] - No studied stack ships a *reusable Go control plane*; only orchestration **patterns** transfer. The microservice decomposition would be greenfield Helix work with no reuse dividend at MVP. [ota_landscape_report §2.3 commercial-oss-fleet Go-fit (2); §3.2] - The synthesis explicitly defers microservices to “at scale,” treating MVP microservices as premature operational cost. [additions_synthesis §5 C2]

2.2 Option B — Modular monolith (extractable seams)

One deployable Go binary. Internal Go package boundaries mirror the future services; the **rollout-engine** and **OS-adapter** are built as cleanly extractable modules with well-defined interfaces. [additions_synthesis §5 C2; master-design §4]

Evidence for. - Directly implements the synthesis resolution of C2: “MVP favors a modular monolith (one deployable, internal package boundaries mirroring the future services) to cut operational cost; the OS-adapter + rollout-engine seams stay extractable.” [additions_synthesis §5 C2] - Honors the decoupling principle (one purpose, well-defined interface, independently testable units) *without* paying per-service deployment, networking, and observability cost at MVP. [master-design §4 (§11.4.28)] - Matches the report's lean-MVP bias: fewest deployables consistent with correctness, especially if a JVM hawkBit is already in the picture. [ota_landscape_report §3.2] - The OS-adapter seam is the studied universality boundary (Android now, Linux later via RAUC/OSTree candidates), so designing it as a module — not a service — keeps Phase-1 lean while preserving the later-phase path. [additions_synthesis §4 (OS-adapter destination — **BROKEN REF, flag for upstream correction**: inherited as 00-master/architecture.md, but no such file exists; the master design file is 2026-06-07-helix-ota-design.md and architecture is its §4); ota_landscape_report §4 RAUC/OSTree “later Linux/universal phase”] - The rollout engine remains independently testable to the floor-≥90% bar required for rollout-gate logic, whether or not it is a separate process. [additions_synthesis §5 C8]

Evidence against. - Requires discipline to keep internal boundaries from eroding; a monolith can decay into Option C without enforcement. (Mitigation in §3.1.) - A single deployable couples the scaling of telemetry ingestion to rollout-control traffic until the split trigger fires; telemetry is plausibly the higher-volume of the two (**UNVERIFIED** — confirm via MVP load tests). (Mitigation: the trigger in §3.2 is defined precisely to catch this.)

2.3 Option C — Unstructured single-binary monolith

One binary with no enforced internal seams (everything in shared packages).

Evidence against. - Violates the decoupling principle requiring separate, independently testable modules (artifact-validator, rollout-engine, ota-protocol, update_engine bridge, OS-adapter). [master-design §4 (§11.4.28)] - Forfeits the

extractability the synthesis requires for the rollout-engine and OS-adapter seams, making the later service split a rewrite rather than a lift-out. [additions_synthesis §5 C2; §4] - Rejected without further analysis; it satisfies neither the MVP-lean goal beyond Option B nor any future-proofing.

3. Decision

Adopt Option B — a modular monolith with extractable seams — for 1.0.0-MVP. The Helix-authored Go control plane ships as **one deployable binary** (Gin, REST-primary, Brotli, HTTP/3→HTTP/2; PostgreSQL + MinIO/S3) with enforced internal module boundaries. The **rollout-engine** and **OS-adapter** are built as cleanly extractable modules. Microservice extraction is deferred until the scale trigger in §3.2 fires. [additions_synthesis §5 C2; master-design §4; ota_landscape_report §3.2]

This decision concerns only Helix-authored code. If ADR-0001 adopts hawkBit, hawkBit remains a **separate deployable behind the Go control plane** (hawkbit-monolith + PostgreSQL + DDI-only at MVP), which is orthogonal to and consistent with this topology. [ota_landscape_report §3.2; eclipse-hawkbit]

3.1 Module boundaries (the seams)

Internal packages mirror the future services and the decoupling principle's named units: [master-design §4 (§11.4.28); additions_synthesis §4]

- **artifact-validator** — server-side SHA-256 + signature verification on upload (defense-in-depth alongside device-side verify-before-apply). [additions_synthesis §3, §6 (file:// verify-before-apply)]
- **rollout-engine** — OS-agnostic staged-rollout state machine (percentage phases gated by success/error thresholds, pause/halt on breach); **designed as an extractable submodule** and the first split candidate. [additions_synthesis §3, §4, §5 C2]
- **ota-protocol types** — shared release/deployment/telemetry contracts. [master-design §4]
- **update_engine bridge** — server side of the thin Android applyPayload client contract. [ota_landscape_report §3.1; master-design §4]
- **OS-adapter** — the universality seam (Android Phase 1; Linux candidates RAUC/OSTree later), built as a module so a future-OS phase is an add, not a rewrite. [additions_synthesis §4; ota_landscape_report §4]

Boundary enforcement (internal-import discipline / module interfaces) is mandatory so the monolith cannot decay into Option C and so each seam is independently testable to its coverage floor. [master-design §4; additions_synthesis §5 C8]

3.2 Scale trigger to split into services

Extract a module into its own service **when any one of the following fires** (first-mover is the rollout-engine, then telemetry ingestion):

1. **Divergent scaling pressure** — telemetry-ingestion load and rollout-control load require materially different horizontal scaling, such that scaling the monolith to satisfy one wastes resources on the other. (Telemetry is an independent event stream from devices feeding dashboards, and is plausibly

higher-volume than rollout control — **UNVERIFIED**; confirm via MVP load tests.) [additions_synthesis §3 (telemetry event stream); §4 (rollout engine as submodule)]

2. **Second OS target enters active development** — when the Linux/universal phase begins, the OS-adapter seam is extracted so Android and Linux delivery paths deploy and scale independently. [ota_landscape_report §metadata (Linux later phase), §4 RAUC/OSTree; additions_synthesis §4 (OS-adapter seam)]
3. **Independent release cadence** — a seam (most likely the rollout-engine) needs to ship on a cadence that the monolith's release train blocks, justifying its own deployable. [additions_synthesis §5 C2 (microservices “revisited at scale”)]

Concrete quantitative thresholds (device count, req/s, telemetry events/s) are **UNVERIFIED** — the cited sources provide no load figures, and the additions drafts' constants are explicitly non-binding. The trigger above is qualitative by necessity; numeric thresholds are to be set from MVP load-test data before the first split and recorded in a follow-up ADR revision. [additions_synthesis §5 C3 (drafts' constants non-binding); §6 (no-guessing); ota_landscape_report §5 (open items)]

Because the seams are extractable by construction (§3.1), satisfying a trigger is a **lift-out**, not a rewrite. [additions_synthesis §5 C2]

4. Consequences

4.1 Positive

- **Lowest MVP operational cost** consistent with correctness — one Helix deployable, aligned with the report's lean-MVP bias and especially valuable if a JVM hawkBit is already running alongside. [ota_landscape_report §3.2, §4 (DMF/RabbitMQ deferral); eclipse-hawkbite §7]
- **Preserves the decoupling principle** — every named unit is a separate, independently testable module, satisfying the architecture mandate and the ≥90% floor on safety-critical rollout-gate/signing/apply paths. [master-design §4 (§11.4.28); additions_synthesis §5 C8]
- **Future-proof without speculation** — the rollout-engine and OS-adapter are extractable, so the later Linux/universal phase and any scale-driven split are lift-outs, not rewrites. [additions_synthesis §5 C2, §4; ota_landscape_report §4]
- **No reuse forfeited** — no studied stack ships a reusable Go control plane, so a microservice decomposition would buy no reuse dividend at MVP; the monolith loses nothing here. [ota_landscape_report §2.3 commercial-oss-fleet Go-fit; §3.2]
- **Catalogue-first compatible** — cross-cutting concerns (auth, observability, eventbus, ratelimiter, cache, http3) come from the verified submodule catalogue and embed cleanly in one binary. [master-design §10 (§11.4.74); additions_synthesis §6, §5 C7]

4.2 Negative

- **Coupled scaling until a trigger fires** — telemetry ingestion and rollout control scale together in the monolith (telemetry is plausibly the higher-volume of the two — **UNVERIFIED**, confirm via MVP load tests); mitigated by trigger #1, which is defined to catch exactly this. [additions_synthesis §3]

- **Boundary-erosion risk** — internal seams require active enforcement or the monolith degrades toward Option C; mitigated by mandated import/interface discipline (§3.1). [master-design §4]
- **Eventual migration cost** — splitting later incurs networking/observability/deployment work that an upfront microservices design would have amortized; accepted as the cost of avoiding premature complexity, and bounded by the extractable-by-construction seams. [additions_synthesis §5 C2]
- **Trigger thresholds unproven at authoring time** — the split trigger is qualitative because no load figures exist in the sources (**UNVERIFIED**); numeric thresholds must be set from MVP load tests. [additions_synthesis §6; ota_landscape_report §5]

5. Status

Proposed. Pending the operator review gate. Ratifies the synthesis's pre-resolution of contradiction **C2**. [additions_synthesis §5 C2; master-design §15 ADR list]

6. Compliance notes (HelixConstitution)

Clause references use the mapping established in the master design's compliance section. [master-design §compliance table]

- **§11.4.28 (decoupling).** The modular-monolith seams (artifact-validator, rollout-engine, ota-protocol, update_engine bridge, OS-adapter) implement the requirement that each unit have one purpose, a well-defined interface, and independent testability. The split trigger (§3.2) preserves this at service granularity. [master-design §4]
- **§11.4.6 (no-guessing) / §11.4.8 (research-before-implementation).** This ADR makes the topology decision on cited evidence rather than draft assertion, and explicitly marks the absence of load figures and the numeric split thresholds as **UNVERIFIED** rather than inventing them. [master-design §2 D3; additions_synthesis §6; §7.1 no-bluff]
- **§11.4.74 (catalogue-first reuse).** Cross-cutting capabilities are taken from the verified submodule catalogue rather than re-implemented, and the single-binary topology hosts them without new infrastructure (Redis only if the cache brick is insufficient — C7). [master-design §10; additions_synthesis §6, §5 C7]
- **§11.4.76 (containers substrate).** The single deployable runs on the containers submodule substrate; a future per-service split reuses the same substrate. [master-design §10; additions_synthesis §4 (deployment)]
- **§1 / §1.1 (four-layer testing + mutation immunity).** Module boundaries keep each seam independently testable; safety-critical paths (signing, apply, rollout-gate) carry the $\geq 90\%$ floor regardless of mono/micro packaging. [master-design §13; additions_synthesis §5 C8]

7. Open / UNVERIFIED items

1. **Numeric split thresholds** (device count, req/s, telemetry events/s) — no load figures exist in the sources; set from MVP load tests and record in a revision. **UNVERIFIED.** [additions_synthesis §6; ota_landscape_report §5]

2. **hawkBit adoption (ADR-0001)** affects how many *non-Helix* deployables exist alongside the monolith; this ADR holds either way but the operational-cost framing depends on that outcome. [ota_landscape_report §3.2; additions_synthesis §7 ADR-0001]
3. **First-split ordering** — rollout-engine is the presumed first extraction, but this should be confirmed against observed MVP load before the split executes. [additions_synthesis §4, §5 C2]
4. **Broken OS-adapter destination (upstream)** — additions_synthesis §4 routes the OS-adapter to 00-master/architecture.md, but no such file exists; the master design file is 2026-06-07-helix-ota-design.md (architecture is its §4). Flag for upstream correction in additions_synthesis §4. [additions_synthesis §4]

8. Sources

- [../ota_landscape_report.md](#) — Helix OTA Landscape Report (locked strategy, mandated stack, §3.2 lean-MVP topology, §4 not-use rationale).
- [../additions_synthesis.md](#) — Reconciliation of operator drafts (§5 C2 topology resolution; C3/C4/C7/C8; §4 destinations; §6 corrections; §7 ADR routing).
- [../00-master/2026-06-07-helix-ota-design.md](#) — Master design (§4 decoupling principle; §10 catalogue/containers; §13 testing; compliance mapping).
- [../stacks/eclipse-hawkbit.md](#) — hawkBit note (monolith deployment option, DMF/RabbitMQ deferral).
- [../stacks/mender.md](#) — Mender note (microservices back end as a contrasting topology and stack conflict).
- [../stacks/rauc.md](#), [../stacks/ostree.md](#) — Linux-phase OS-adapter candidates informing the OS-adapter split trigger.